CVW Inventory Monitor

Computer Vision Walmart (CVW): detección y monitoreo de quiebres de stock en góndolas mediante detección de objetos.

Proyecto Capstone de Ingeniería en Informática, Duoc UC, desarrollado para Walmart Chile.

> **Estado del documento:** avance intermedio, octubre de 2026. Las secciones marcadas como *pendiente* se completarán antes de la entrega final.

---

1. Problema y objetivo

La falta de productos en góndola es una de las principales fuentes de venta perdida en el comercio masivo. Muchas veces el producto está en bodega, pero nadie detecta a tiempo que se agotó en la sala.

CVW busca automatizar esa inspección visual: analiza imágenes de góndolas con un modelo de visión por computador, calcula qué proporción del espacio está vacía y genera una alerta cuando una góndola cae a un nivel crítico de desabastecimiento. Lo hace sin sensores IoT ni básculas en los estantes.

---

2. Estrategia técnica

Detección de espacios vacíos (*void detection*). En lugar de reconocer cada producto (miles de SKU distintos), el modelo distingue solo dos clases:

Clase	Significado
`missing`	Espacio vacío en la góndola
`product`	Producto presente

Con el conteo de ambas clases se calcula la tasa de vacío de la góndola:

```
tasa de vacío = missing / (product + missing)
```

Modelo YOLOv8. Se usa la familia YOLOv8 de Ultralytics, que detecta objetos en una sola pasada de la red neuronal sobre imágenes de 640 × 640 píxeles.

- La primera versión usó **YOLOv8n (nano)**, con unos 3 millones de parámetros.
- La versión actual usa **YOLOv8s (small)**, con unos 11 millones de parámetros. El cambio se hizo porque la versión nano se estancó en un mAP50-95 cercano a 0,38 en validación, y porque se espera incorporar más imágenes al dataset, que un modelo con más capacidad puede aprovechar mejor.

---

3. Justificación del enfoque

- **Menos etiquetado.** Detectar dos clases requiere mucho menos trabajo de etiquetado que reconocer cada producto.
- **Velocidad.** YOLO procesa cada imagen en una sola pasada; con YOLOv8s la inferencia toma entre 9 y 20 ms por imagen en una GPU Tesla T4.
- **Cajas en vez de polígonos.** Se priorizó la detección con cajas delimitadoras sobre la segmentación por simplicidad y velocidad; las cajas bastan para calcular la tasa de vacío.
- **Sin hardware adicional.** No requiere sensores ni básculas, que tienen costo y problemas de calibración.

---

4. Arquitectura y flujo

El sistema es una aplicación web con un backend en Python (FastAPI) y un frontend en React. Cada imagen pasa por cuatro servicios:

```
Fuente de imágenes  (cámara web, cámara IP por RTSP o simulador)
        ↓
Servicio de detección  (modelo YOLOv8 + filtro de región de interés)
        ↓
Servicio de inventario  (conteo de product y missing, tasa de vacío)
        ↓
Servicio de alertas  (umbral, ventana de tiempo, cooldown, registro)
```

Servicio	Archivo	Función
Cámara	`backend/app/services/camera_service.py`	Obtiene las imágenes. Incluye un **simulador de góndola** para probar el sistema sin cámara
Detección	`backend/app/services/detection_service.py`	Ejecuta el modelo y dibuja las cajas (verde: `product`, rojo: `missing`)
Inventario	`backend/app/services/inventory_service.py`	Calcula la tasa de vacío, con protección contra división por cero
Alertas	`backend/app/services/alert_service.py`	Decide cuándo alertar, guarda una captura y registra el evento

Regla de alerta

Se dispara una alerta cuando la tasa de vacío es **igual o mayor a 80 %** de forma sostenida durante **3 segundos**. Después de cada alerta hay un **cooldown de 30 segundos** para no repetir avisos. Un 80 % de vacío equivale a un 20 % de stock visible, el nivel crítico definido en el proyecto.

Cada alerta guarda una imagen en `alerts/images/` y un registro en `alerts/logs/alerts.jsonl`. El código deja preparado el envío a un *webhook* externo.

Mejora continua del modelo

El sistema incluye un ciclo de *active learning*:

1. Guarda automáticamente las imágenes donde el modelo tiene baja confianza o detecta demasiado vacío.

2. Una persona revisa y corrige esas imágenes desde la aplicación.
3. Con las imágenes aprobadas se reentrena el modelo en segundo plano.
4. El modelo nuevo se evalúa y, si cumple el criterio (mAP50 ≥ 0,80), puede reemplazar al actual sin detener el sistema. Si algo falla, se puede volver a la versión anterior.

---

5. Datos

Aspecto	Detalle
Fuente	Roboflow Universe, dataset "Empty Spaces in a Supermarket Hanger", versión 4
Tamaño	349 imágenes de góndolas (la mayoría de 720 × 960 px)
Etiquetado	53.278 cajas: 51.862 `product` y 1.416 `missing`
Densidad	En promedio, 149 productos y 4 espacios vacíos por imagen (máximo: 501 productos)
Origen	Dataset público; no proviene de salas de Walmart

Preparación

Partición	Imágenes originales	Cajas `missing`	Cajas `product`	Tratamiento
Train	243 (69,6 %)	994	35.978	+ 1.352 copias aumentadas = 1.595 imágenes
Valid	53 (15,2 %)	211	7.463	Imágenes limpias
Test	53 (15,2 %)	211	8.421	Imágenes limpias
Valid_cam / Test_cam	53 + 53	211 / 211	7.463 / 8.421	Copias degradadas que simulan cámara: compresión, baja resolución, ruido, sombras y modo monocromo

- **Aumento de datos.** Solo en train: cambios de perspectiva, rotación, distorsión de lente y degradación de cámara. Las imágenes que contienen `missing` generan 6 copias y el resto 3, para compensar el desbalance.
- **Fuga de datos mitigada.** Las imágenes casi idénticas se agrupan con *hashes* perceptuales antes de dividir el dataset, para que una misma góndola no quede a la vez en entrenamiento y en prueba. Se encontraron 339 grupos; el mayor tiene 2 imágenes.
- **Partición equilibrada.** Entre 300 particiones posibles se eligió la que mejor respeta el 70/15/15 tanto en imágenes como en cajas `missing`.
- **Reproducibilidad.** Las particiones, los aumentos y el entrenamiento usan una semilla fija (`SEED = 42`).

Desafíos detectados

- **Desbalance de clases.** Hay unas 37 cajas `product` por cada caja `missing`.
- **Objetos pequeños.** El 5,6 % de los productos mide menos de 8 píxeles a 640 px, por debajo del paso mínimo de la grilla de YOLO, y el 42,5 % mide menos de 16 píxeles.
- **Un vacío no tiene forma propia.** Es solo el fondo de la góndola expuesto, lo que lo hace más difícil de detectar que un producto.

6. Resultados

YOLOv8s (versión actual)

Entrenamiento: 100 épocas, 640 px, batch 16, en Google Colab con GPU Tesla T4 (46 minutos). Modelo resultante: `best.pt`, 22,5 MB.

Métrica	Valid	Valid cámara	Test	Test cámara
Precisión	73,8 %	69,2 %	69,2 %	63,5 %
Recall	64,2 %	62,7 %	62,9 %	56,8 %
mAP50	68,0 %	63,7 %	**63,3 %**	**58,2 %**
mAP50-95	37,5 %	31,8 %	**33,6 %**	**28,1 %**
`missing` mAP50	51,8 %	49,7 %	45,6 %	43,8 %
`missing` recall	50,2 %	52,8 %	47,9 %	43,7 %
`product` mAP50	84,2 %	77,8 %	81,0 %	72,7 %
`product` recall	78,1 %	72,5 %	77,9 %	70,0 %

- **Test** (53 imágenes nunca vistas) es el resultado oficial; **test cámara** estima el rendimiento con una cámara de seguridad.
- **Productos:** buen desempeño, con un mAP50 de 81,0 % (72,7 % con degradación).
- **Espacios vacíos:** es la principal limitación; el modelo detecta cerca de la mitad de los huecos reales.
- **Efecto de la calidad de imagen:** la degradación reduce el mAP50-95 en 0,055, una caída moderada que indica que la aumentación simulada cumple su función.
- **Umbral de confianza recomendado:** 0,38, el que maximiza el F1 en validación (F1 = 0,688; `missing` 0,551; `product` 0,826).

Comparación con la versión anterior (YOLOv8n)

Versión	Dataset	Evaluación	mAP50	mAP50-95
v1 · YOLOv8n	Roboflow v3, ~200 imágenes	Validación, partición aleatoria	72,8 %	38,8 %
v2 · YOLOv8s	Roboflow v4, 349 imágenes	Test, partición sin casi-duplicados	63,3 %	33,6 %

Las cifras **no son directamente comparables**: cambian el dataset, el conjunto de evaluación y la forma de dividir las imágenes. La evaluación de la v2 es más estricta, porque usa imágenes nunca vistas y evita que una misma góndola quede en entrenamiento y en prueba.

Experimento con SAM

Se probó Segment Anything (SAM) para refinar el área de los espacios vacíos. Las máscaras no colapsaron (2 %) y se salen poco de la caja (10 %), pero cada imagen tarda unos 500 ms. El resultado es **no concluyente**: para decidir si mejora el área real se necesitan polígonos dibujados a mano en unas 20 imágenes.

Casos destacados

Pendiente: análisis de los 10 mejores y 10 peores casos en test cámara.

---

7. Estructura del repositorio

```
├── backend/
│   ├── app/
│   │   ├── api/        # Rutas de la API (cámara, inventario, alertas, modelos, dataset,
entrenamiento)
│   │   ├── core/       # Configuración y registro de eventos
│   │   ├── ml/         # Carga, cambio y evaluación de modelos
│   │   ├── services/   # Cámara, detección, inventario, alertas, dataset y
reentrenamiento
│   │   ├── schemas/    # Validación de datos
│   │   └── main.py     # Aplicación principal
│   └── tests/          # Pruebas automáticas (pytest)
├── frontend/           # Aplicación web (React + TypeScript + TailwindCSS)
├── models/
│   ├── production/     # Modelo activo (best.pt + metadata.json)
│   ├── candidates/     # Modelos reentrenados en espera de aprobación
│   └── archive/        # Versiones anteriores
├── data/               # Muestras recolectadas, en revisión y aprobadas
├── alerts/             # Capturas y registro de alertas
├── configs/
│   └── app_config.yaml # Parámetros del sistema
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── run.py              # Punto de arranque
```

---

8. Instalación y ejecución

Requisitos

- Python 3.10 o superior
- Node.js 18 o superior (para compilar el frontend)
- Recomendado: GPU NVIDIA con CUDA. Sin GPU, el sistema funciona en CPU.

Pasos

```
        # 1. Descargar el proyecto
        git clone https://github.com/Linzdhx/Capstone.git
        cd Capstone

        # 2. Instalar dependencias de Python
        python -m pip install -r requirements.txt

        # 3. Compilar el frontend
        cd frontend
        npm install
        npm run build
        cd ..
```

```
# 4. Crear el archivo de configuración local
cp .env.example .env

# 5. Iniciar el sistema
python run.py
```

Luego abrir en el navegador:

- Aplicación: http://localhost:8000
- Documentación de la API: http://localhost:8000/docs

Probar sin cámara

En el archivo `.env`, usar `CAMERA_SOURCE_TYPE=synthetic`. El sistema genera una góndola simulada para probar detecciones, cálculo y alertas sin hardware.

Con Docker

```
docker compose up --build -d
```

Pruebas automáticas

```
python -m pytest backend/tests -v
```

Las 12 pruebas verifican el cálculo de inventario (incluido el caso sin detecciones), el filtro de región de interés, la ventana de 3 segundos, el cooldown, la gestión de modelos y el ciclo de vida de las muestras.

---

9. Parámetros principales

Se ajustan en `configs/app_config.yaml` o en `.env`:

Parámetro	Valor por defecto	Descripción
`empty_threshold`	0,80	Tasa de vacío que activa la alerta
`consecutive_seconds`	3,0	Segundos que debe mantenerse la condición
`cooldown_seconds`	30,0	Espera mínima entre alertas
`default_confidence`	0,25	Confianza mínima de una detección
`default_iou`	0,45	Umbral para eliminar cajas duplicadas
`source_type`	webcam	Fuente: `webcam`, `usb`, `rtsp` o `synthetic`

---

10. Estado actual y próximos pasos

Pendientes del avance

- Subir al repositorio los pesos de YOLOv8s (`best.pt`, 22,5 MB); el `best.pt` actual del repositorio corresponde a YOLOv8n.
- Subir el notebook de entrenamiento v2, para que las particiones y el entrenamiento sean reproducibles.
- Completar el análisis de los 10 mejores y 10 peores casos.

Mejoras propuestas

Acción	Descripción	Impacto esperado
Aumentar la resolución	Pasar de 640 a 800 o 960 píxeles	Detectar productos menores a 8 px
SAHI	Analizar la imagen por recortes superpuestos	Mejor detección en estantes densos
Validar SAM	Dibujar polígonos en ~20 imágenes y comparar el área con la caja	Decidir si SAM justifica sus ~500 ms por imagen
Más imágenes	Ampliar el dataset, idealmente con imágenes reales de sala	Mejor generalización

---

Equipo

David Salvatierra · David Yáñez · Francisco Fernández · Francisco Merino